

Phát triển ứng dụng web

NodeJS

Nội dung

- ☐ Node.js
- ☐ Express.js
- ☐ MVC pattern

Nội dung

- ☐ **Node.js**
- ☐ Express.js
- ☐ MVC pattern

Node.js

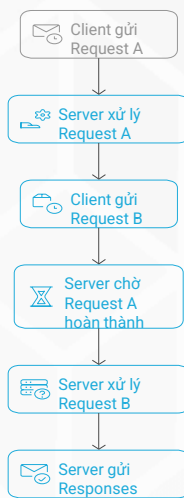
- ☐ Node.js - Đưa JavaScript ra khỏi trình duyệt.
 - ☐ Không phải là một ngôn ngữ lập trình mới.
 - ☐ Không phải là một framework.
- ☐ Node.js là:
 - ☐ Một **môi trường thực thi (runtime)** JavaScript phía máy chủ (server-side).
 - ☐ Nó sử dụng **V8 Engine** (của Google Chrome) để biên dịch và chạy code JS.
 - ☐ Cho phép JavaScript làm những việc mà trước đây chỉ C#, Java, Python... làm được (đọc/ghi file, kết nối CSDL, tạo web server...).

Đặc trưng của Node.js

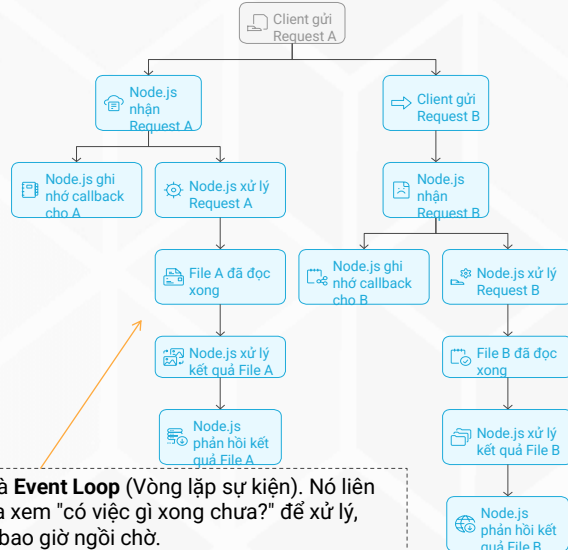
- ❑ **Asynchronous and Non-blocking I/O Mode:** những **task** có liên quan về **I/O** được xử lý bất đồng bộ và không có quyền ngăn chặn những **task** khác giúp tối ưu hóa khả năng phản hồi của ứng dụng khi xử lý đồng thời nhiều yêu cầu.
- ❑ **Thực thi nhanh:** nhờ nền tảng **Javascript Engine V8**.
- ❑ **Scalability:** **Node.js** được thiết kế chủ yếu để xử lý nhiều kết nối đồng thời, làm cho nó phù hợp với các ứng dụng có khả năng mở rộng như ứng dụng thời gian thực và ứng dụng chat.
- ❑ **Cross-Platform:** **Node.js** hỗ trợ nhiều hệ điều hành khác nhau, giúp dễ dàng chuyển đổi giữa các môi trường.

Event Loop & Non-Blocking I/O

Xử lý Blocking



Xử lý Non-Blocking



Đây chính là **Event Loop** (Vòng lặp sự kiện). Nó liên tục kiểm tra xem "có việc gì xong chưa?" để xử lý, chứ không bao giờ ngồi chờ.

Node.js - Module

- ❑ Trong **Node.js**, một **module** là một đoạn mã có thể tái sử dụng, đóng gói các chức năng liên quan. Các **module** giúp tổ chức mã, làm cho mã dễ bảo trì và mở rộng hơn. **Node.js** sử dụng một hệ thống **module** dựa trên đặc tả **module CommonJS**. Mỗi tệp trong **Node.js** được coi như một **module** riêng biệt.
- ❑ Các loại **module** phổ biến trong **Node.js** :
 - ❑ Core Modules
 - ❑ NPM (Node Package Manager) Modules
 - ❑ User-Defined Modules
 - ❑ Native Addons

CommonJS

- ❑ Các **module CommonJS** là tiêu chuẩn trong **Node.js** cho đến khi các **module ECMAScript** được hỗ trợ. **CommonJS** đã là nền tảng chính để quản lý các **dependency** và tổ chức mã thành các **module**.
- ❑ Các **module** được **export** bằng cách sử dụng **module.exports** và được nhập vào bằng cách sử dụng hàm **require()**.

Đặc trưng của CommonJS module

- ❑ **Synchronous Loading:** Các **module CommonJS** được tải theo cách đồng bộ. Điều này hoạt động tốt trên phía **server**, nơi các **module** được **load** từ cục bộ, nhưng lại kém hiệu quả hơn trên **web**, nơi **load** bất đồng bộ được ưu tiên để giảm thời gian chờ.
- ❑ **Non-browser Support:** Ban đầu, **CommonJS** không được hỗ trợ nguyên bản trong các trình duyệt **web**. Các công cụ như **Webpack** hoặc **Browserify** là cần thiết để đóng gói các **module** này cho việc sử dụng trên trình duyệt.
- ❑ **Dynamic:** Có thể **load** các **module** động và tạo chúng trong thời gian chạy (**runtime**). Hàm **require()** có thể được gọi một cách có điều kiện trong các khối mã.

ES Modules (ECMAScript modules)

- ❑ **ES Modules** đã được **Node.js** hỗ trợ chính thức từ phiên bản 12, với hỗ trợ thử nghiệm trước đó bắt đầu từ phiên bản 8.5.
- ❑ Các tệp có phần mở rộng **.mjs** được coi là các **ES modules**, hoặc có thể thiết lập **"type": "module"** trong tệp **package.json** để coi các tệp **.js** là ES modules.

Đặc trưng của ES Modules

- ❑ **Asynchronous Loading: ES Modules** hỗ trợ tải bất đồng bộ, điều này mang lại lợi ích cho các ứng dụng web, giảm thời gian tải ban đầu và cải thiện hiệu suất web.
- ❑ **Browser Support: ES Modules** được các trình duyệt **web** hiện đại hỗ trợ sẵn, giúp việc chia sẻ **module** giữa **server** và **client** trở nên dễ dàng hơn mà không cần các công cụ bổ sung.
- ❑ **Static Structure:** Việc **import** và **export** phải được đặt ở cấp cao nhất của tệp. Không giống như **CommonJS**, không thể import **ES Modules** có điều kiện vào trong hàm hay các khối lệnh.

Sự tương tác

- ❑ **Node.js** cho phép tương tác giữa các **module CommonJS** và **ES Modules**.
 - ❑ Các **module CommonJS** có thể được import vào trong **ES Modules** bằng câu lệnh import mặc định (**import pkg from 'cjs-package'**);).
 - ❑ Tuy nhiên việc import **ES Modules** vào **CommonJS** thì yêu cầu sử dụng các câu lệnh động **import()** bởi **require()** không thể hiểu **ES Module** syntax.
- ❑ Cả hai hệ thống **module** đều phục vụ tốt các mục đích riêng trong những điều kiện khác nhau. Việc lựa chọn giữa chúng nên dựa trên nhu cầu cụ thể của ứng dụng, bao gồm các yêu cầu về hiệu suất và môi trường mà ứng dụng hoạt động.

CommonJS vs. ES Modules

Tiêu chí	CommonJS (CJS)	ES Modules (ESM)
Cú pháp Import	<code>const myModule = require('my-module');</code>	<code>import myModule from 'my-module';</code>
Cú pháp Export	<code>module.exports = myFunction;</code>	<code>export default myFunction;</code>
Tải Module	Đồng bộ (Synchronous)	Bất đồng bộ (Asynchronous)
Thời điểm chạy	require là một hàm, chạy tại chỗ	import là cú pháp, được xử lý (parsed) trước khi chạy
Nơi sử dụng	Node.js (truyền thống)	Node.js mới & Trình duyệt (Browser)
this ở cấp cao nhất	module.exports (hoặc exports)	undefined
Chế độ nghiêm ngặt	Không mặc định	Luôn luôn ('use strict')
File mặc định	.js (nếu không khai báo)	.mjs, hoặc .js (nếu có "type": "module")

Lựa chọn CJS hay ESM

- ❑ **CommonJS (CJS):** Dùng khi đang bảo trì các dự án Node.js cũ, hoặc khi phụ thuộc vào các thư viện cũ chưa hỗ trợ ESM.
- ❑ **ES Modules (ESM):** Nên là lựa chọn mặc định cho tất cả các dự án mới.
 - ❑ Đây là tiêu chuẩn JavaScript toàn cầu.
 - ❑ Code có thể tái sử dụng giữa Node.js và trình duyệt.
 - ❑ Nó cho phép các công cụ build (như Webpack, Vite) thực hiện "tree-shaking" (loại bỏ code chết) hiệu quả hơn vì cấu trúc tĩnh của nó.

Ví dụ: CJS Module - exports

myPI.js

```
module.exports = 3.14156;
```

MyFunc.js

```
module.exports = n => {  
  return Math.round(Math.random() * n);  
};
```

main.js

```
const pi = require('./myPI');  
console.log(`The value of PI is ${pi}`);  
  
const func = require('./myFunc');  
const n = 1000;  
console.log(`Random < ${n} = ${func(n)}`);
```

Ví dụ: CJS Module – exports object

myCalculator_1.js

```
module.exports = {  
  add: (a, b) => {  
    return a + b;  
  },  
  subtract: (a, b) => {  
    return a - b;  
  }  
};
```

myCalculator_2.js

```
exports.add = (a, b) => {  
  return a + b;  
};  
  
exports.subtract = (a, b) => {  
  return a - b;  
};
```

main.js

```
const cc1 = require('./myCalculator_1');  
const cc2 = require('./myCalculator_2');  
const a = 6, b = 11;  
console.log(`${a} + ${b} = ${cc1.add(a, b)} / ${cc2.add(a, b)}`);  
console.log(`${a} - ${b} = ${cc1.subtract(a, b)} / ${cc2.subtract(a, b)}`);
```

Ví dụ: ES Modules

```
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
export default {
  add,
  subtract
};
export { add, subtract };
```

myCalculator.mjs

```
import cc from './myCalculator.mjs';
const a = 6, b = 11;
console.log(`${a} + ${b} = ${cc.add(a, b)}`);
console.log(`${a} - ${b} = ${cc.subtract(a, b)}`);
```

main.js

```
import { add, subtract } from './myCalculator.mjs';
const a = 6, b = 11;
console.log(`${a} + ${b} = ${add(a, b)}`);
console.log(`${a} - ${b} = ${subtract(a, b)}`);
```

Node Package Manager

- ❑ **Node Package Manager (NPM)**: là trình quản lý **package** cho **JavaScript**, chủ yếu được sử dụng để quản lý và phân phối các gói (thư viện, công cụ và framework) được viết bằng **JavaScript**. **NPM** đi kèm với **Node.js** và được **install** tự động khi **install Node.js**. Một **package** chứa tất cả các file cần thiết cho 1 **module**.
- ❑ **NPM** cung cấp 2 chức năng chính:
 - ❑ Kho **package** (free): npmjs.com
 - ❑ Tiện ích dòng lệnh (**command**) để cài đặt các **module**, **package** **Node.js** quản lý phiên bản và sự phụ thuộc của các **package**.

NPM - Command

☐ Cài đặt toàn cục

```
npm install -g <package name>
```

☐ Cài đặt chỉ phục vụ cho việc phát triển

```
npm install --save-dev <package name>
```

☐ Lựa chọn phiên bản muốn cài đặt

```
npm install <package name>@<version>
```

☐ Gỡ bỏ

```
npm uninstall <package name>
```

☐ Cập nhật

```
npm update <package name>
```

Module - HTTP

```
import http from 'http';

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain; charset=UTF-8');
  res.end('Hello, World!\n');
});

const port = 3000;
const hostname = '127.0.0.1';
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Module - URL

- ❑ Giúp phân giải và phân tích **URL**.

Property	Mô tả
href	đường dẫn URL
host	tên server và port
hostname	tên server
port	port
path	phần đường dẫn URL gồm cả query
pathname	phần đường dẫn URL
auth	
protocol	
query	phần query
search	

Ví dụ

```
import http from 'http';
import url from 'url';
import cc from './myCalculator_2.js';

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=UTF-8' });
  const q = url.parse(req.url, true).query;
  const a = parseInt(q.a); b = parseInt(q.b);
  res.end(`${a} + ${b} = ${cc.add(a, b)}`);
});
const port = 3000;
const hostname = '127.0.0.1';
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

http://127.0.0.1:3000/?a=12&b=23



12 + 23 = 35

Module – xử lý file

- ❑ **NodeJs** cung cấp **module fs** để có thể truy cập hệ thống file. **Module fs** chịu trách nhiệm cho tất cả các hoạt động **I/O** đồng bộ hoặc bất đồng bộ.
- ❑ **Module path** cung cấp rất nhiều chức năng hữu ích để truy cập và tương tác với hệ thống **file**.

Ví dụ

```
import fs from 'fs';
import path from 'path';

const pathFile = path.join(__dirname, 'data.txt');
fs.readFile(pathFile, 'utf-8', (err, data) => {
  if (err) {
    console.log(err);
  } else {
    console.log(data);
  }
});
```

Sử dụng Promises

```
import { readFile, writeFile } from 'fs/promises';
import path from 'path';
const pathFileIn = path.join(__dirname, 'dataIn.json');
const pathFileOut = path.join(__dirname, 'dataOut.json');
async () => {
  try {
    const rawDT = await readFile(pathFileIn, { encoding:
'utf-8' });
    const dt = JSON.parse(rawDT);
    dt.add = 'new';
    const rawWr = JSON.stringify(dt, null, 3);
    const bufData = Buffer.from(rawDT, 'utf-8');
    await writeFile(pathFileOut, bufData);
  } catch (error) {
    console.log(error);
  }
};
```

Module - EventEmitter

- ❑ **Module events** cung cấp lớp đối tượng **EventEmitter** cho phép kết hợp lập trình hướng sự kiện trong **NodeJs**.

```
import events from 'events';
const eventEmitter = new events.EventEmitter;

function addNumHandler(n) {
  console.log(n);
}
eventEmitter.on('addNum', addNumHandler);
for (let i = 0; i < 5; i++) {
  eventEmitter.emit('addNum', i * i);
}
eventEmitter.removeAllListeners('addNum');
```

Nội dung

- ☐ *Node.js*
- ☐ **Express.js**
- ☐ MVC pattern

Express framework

- ☐ **Express** là một **framework web Node.js** phổ biến nhất và là thư viện cơ bản cho một số **framework Node.js** phổ biến khác.
- ☐ **Express** cung cấp các chức năng chính:
 - ☐ **Routing**: **Express.js** cho phép định nghĩa các **route** để xử lý các phương thức **HTTP** và **URL** khác nhau, giúp dễ dàng tổ chức logic cho ứng dụng.
 - ☐ **Middleware**: **Express** sử dụng các hàm **middleware** để thực hiện nhiều tác vụ khác nhau trong chu kỳ **request-response**.
 - ☐ **Template Engines**: **Express** hỗ trợ nhiều **template engine** như **EJS**, **Pug** và **Handlebars**, cho phép tạo **HTML** động từ phía **server**.
 - ☐ **Error Handling**: **Express** cung cấp một cơ chế xử lý lỗi mạnh mẽ để xử lý các lỗi có thể xảy ra trong quá trình thực thi ứng dụng.

Ví dụ

```
import express from 'express';
const app = express();
import cc from './myCalculator_2.js';

app.get('/', (req, res) => {
  const a = parseInt(req.query.a); b = parseInt(req.query.b);
  res.end(`${a} + ${b} = ${cc.add(a, b)}`);
});

const port = 3000;
const hostname = '127.0.0.1';
app.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Express – basic routing

- ❑ **Routing** tức là cách ứng dụng hồi đáp cho yêu cầu của **client** đến 1 **endpoint** cụ thể đó là 1 **URL** với **HTTP request method**.
- ❑ Cơ bản xử lý thông qua cấu trúc

```
app.METHOD(PATH, HANDLER)
```

- ❑ **app** là express instance
- ❑ **METHOD** là HTTP request method (lowercase)
- ❑ **PATH** là path server
- ❑ **HANDLER** là callback function.

Express – router paths, params

- ❑ Có thể sử dụng **pattern, regular expression** để kết hợp tạo **path** cho **router**. Ví dụ:
 - ❑ 'ab?cd' → acd, abcd
 - ❑ 'ab+cd' → abcd, abbcd, abbbcd,...
 - ❑ 'a(bc)?d' → ad, abcd
- ❑ Sử dụng **params** để truyền tham số trong **request URL**.
 - ❑ Route path: `/users/:userId/books/:bookId`
 - ❑ Request URL: `<hostname>/users/34/books/8989`
 - ❑ req.params: `{ "userId": "34", "bookId": "8989" }`

Express – Static resources

- ❑ Sử dụng hàm tích hợp sẵn **express.static** để cung cấp tài nguyên tĩnh như image, css,...

```
express.static(root, [options])
```

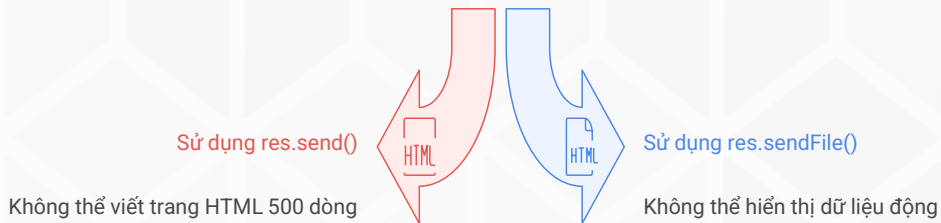
- ❑ Ví dụ

```
app.use(express.static(__dirname + '/public'));
```

```
app.use('/public', express.static(__dirname + '/public'));
```

Response HTML

Làm thế nào để tạo response cho HTTP request?



Template Engines - "Khuôn đúc" HTML

Template Engine

Tạo Template
Tạo các file "mẫu" (template)

Render
"Bơm" dữ liệu vào các biến



Express – Template Engine

- ❑ **Express.js** cung cấp cách thức để xây dựng **template engine** giúp việc tạo mã **HTML** hồi đáp cho **client** dễ dàng hơn.
 - ❑ Sử dụng `app.engine()` để xây dựng template.
 - ❑ Sử dụng `app.set('views', <path>)` để thiết lập nơi chứa file template.
 - ❑ Sử dụng `app.set('view engine', <template name>)` để thiết lập sử dụng template.
 - ❑ Sử dụng `response.render()` để sử dụng template tạo mã **HTML** hồi đáp cho client.

Ví dụ

```
app.engine('html', (filePath, options, callback) => { // define the template engine
  fs.readFile(filePath, 'utf-8', (err, content) => {
    if (err) return callback(err);
    // this is an extremely simple template engine
    const rendered = content.toString()
      .replace('{{title}}', `<title>${options.title}</title>`)
      .replace('{{msg}}', `<h1>${options.msg}</h1>`);
    return callback(null, rendered);
  });
});
```

```
app.set('views', './views') // specify the views directory
app.set('view engine', 'html') // register the template engine
```

```
res.render('home', { title: 'template', msg: 'Hello Tpl' });
```

View Template Engine - Handlebars

❑ Cấu hình với **express-handlebars**

```
import { create } from 'express-handlebars';
// Configure Handlebars
const hbs = create({
  extname: '.hbs',
  layoutsDir: 'views/layouts/',
  defaultLayout: 'main',
});
// Register Handlebars as the view engine
app.engine('.hbs', hbs.engine);
app.set('view engine', '.hbs');
app.set('views', './views');
```

Vấn đề: Lặp lại Code HTML (WET)

index.hbs

```
<!DOCTYPE html>
<html>
<head>
  <title>{{title}}</title>
  <link rel="stylesheet"
href="/css/style.css">
  <script
src="/js/main.js"></script>
</head>
<body>
  <nav>...</nav>

  <h1>Trang chủ</h1>
  <p>Đây là nội dung trang
chủ.</p>

  <footer>...</footer>
</body>
</html>
```

about.hbs

```
<!DOCTYPE html>
<html>
<head>
  <title>{{title}}</title>
  <link rel="stylesheet"
href="/css/style.css">
  <script
src="/js/main.js"></script>
</head>
<body>
  <nav>...</nav>

  <h1>Trang giới thiệu</h1>
  <p>Đây là nội dung trang giới
thiệu.</p>

  <footer>...</footer>
</body>
</html>
```

Sức mạnh của Layouts (DRY!)

- ❑ Là một file template "**cha**" (master template).
- ❑ Nó định nghĩa cấu trúc HTML chung (khung sườn) của toàn bộ trang web.
- ❑ Nó có một "lỗ hổng" (placeholder) đặc biệt để "nhét" nội dung của các trang "con" vào.
 - ❑ Trong **hbs**, **placeholder** đó mặc định là `{{body}}`

Middleware trong Express.js

- ❑ Cơ bản khi **Express.js** hoạt động là các **Middleware** được thực hiện liên tiếp nhau từ quá trình nhận **request** cho đến **response**.
- ❑ Tham số cơ bản cho **Middleware** trong **Express.js** là:
 - ❑ **req**: Request
 - ❑ **res**: Response
 - ❑ **next**: để chuyển sang **Middleware** tiếp theo.
 - ❑ **err**: quản lý lỗi.

Chức năng middleware trong express.js

- ❑ Thực hiện bất cứ đoạn code nào.
 - ❑ Làm thay đổi các đối tượng **request** và **response**.
 - ❑ Kết thúc một quá trình **request - response**.
 - ❑ Gọi hàm **middleware** tiếp theo trong stack.
- ☞ Nếu hàm **middleware** hiện tại không kết thúc quá trình **request - response** thì nó phải gọi hàm **next()** để chuyển sang **middleware** tiếp theo.

Ví dụ

```
import 'dotenv/config';
import express from 'express';
const app = express();
const port = process.env.PORT | 3000;

app.use(function(req, res, next){
  req.data = 'abc';
  next();
});

app.use(express.static(__dirname + '/public'));

app.get('/abc', (req, res) => {
  res.end(req.data);
});
```

Các loại Middleware trong Express.js

- ☐ **Application-level middleware** (middleware cấp ứng dụng).
- ☐ **Router-level middleware** (middleware cấp điều hướng - router).
- ☐ **Error-handling middleware** (middleware xử lý lỗi).
- ☐ **Built-in middleware** (middleware sẵn có).
- ☐ **Third-party middleware** (middleware của bên thứ ba).

Nội dung

- ☐ *Node.js*
- ☐ *Express.js*
- ☐ **MVC pattern**

Vấn đề khi quy mô ứng dụng tăng

```
// app.js
app.get('/users', (req, res) => {
  // 1. Code kết nối CSDL...
  // 2. Viết câu truy vấn SQL/NoSQL...
  // 3. Xử lý logic nghiệp vụ...
  // 4. Định dạng dữ liệu...
  // 5. Gửi response...
});

app.post('/users', (req, res) => {
  // 1. Code validate dữ liệu...
  // 2. Code kết nối CSDL...
  // 3. Viết câu lệnh INSERT...
  // 4. Xử lý lỗi...
  // 5. Gửi response...
});

// ... 50 routes khác cũng y như vậy ...
```

MVC pattern

❑ **Model-View-Controller (MVC)** là một **mẫu kiến trúc phần mềm (architectural pattern)**. Nó chia một ứng dụng thành ba thành phần chính, có liên kết với nhau:

- ❑ **Model:** Dữ liệu và logic nghiệp vụ.
- ❑ **View:** Giao diện người dùng (UI).
- ❑ **Controller:** Xử lý yêu cầu (input) và điều phối.

Model: "Bộ não" của ứng dụng

- ☐ **Model:** Đại diện cho **dữ liệu** và **logic nghiệp vụ (business rules)** của ứng dụng.
- ☐ **Trách nhiệm:**
 - ☐ Định nghĩa cấu trúc dữ liệu.
 - ☐ Chứa các quy tắc nghiệp vụ.
 - ☐ Tương tác trực tiếp với cơ sở dữ liệu (Database).
- ☐ **Model KHÔNG làm gì:**
 - ☐ **Không** biết gì về HTML, CSS.
 - ☐ **Không** quyết định cách hiển thị dữ liệu.
 - ☐ **Không** xử lý trực tiếp HTTP Request.

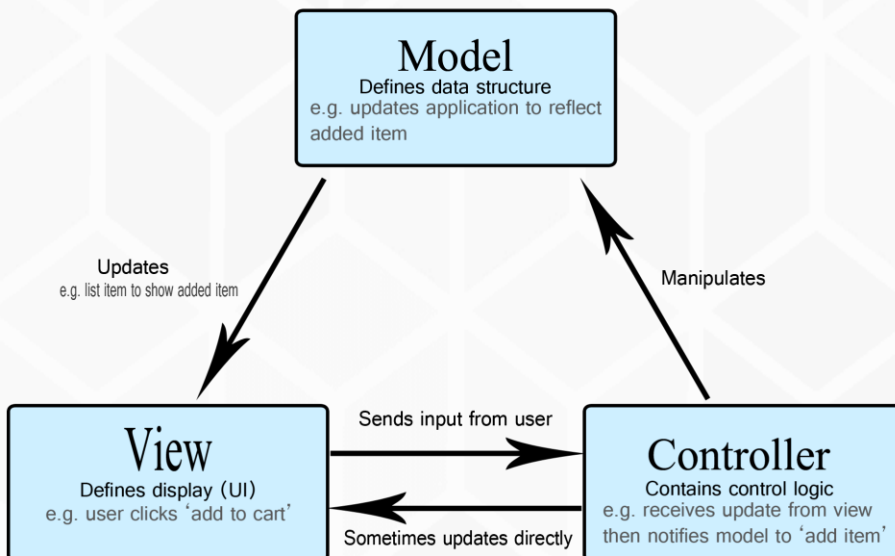
View: "Bộ mặt" của ứng dụng

- ☐ **View:** Là **giao diện người dùng (UI)**. Là những gì người dùng cuối **nhìn thấy và tương tác**.
- ☐ **Trách nhiệm:**
 - ☐ Hiển thị dữ liệu mà **Controller** cung cấp.
 - ☐ Gửi hành động của người dùng (click, submit form) đến **Controller**.
- ☐ **Đặc điểm quan trọng:** View "Thụ động" (Dumb View)
 - ☐ View **không** tự ý lấy dữ liệu từ Model.
 - ☐ View **không** chứa logic nghiệp vụ. Nó chỉ hiển thị những gì được bảo.

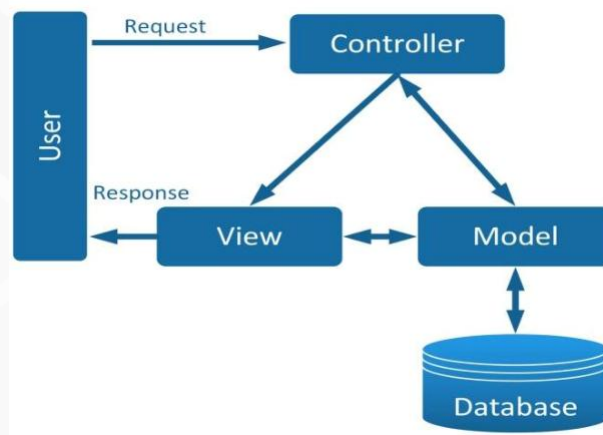
Controller: "Người điều phối"

- ❑ **Controller:** Là "**người trung gian**" đứng giữa Model và View.
- ❑ **Trách nhiệm:**
 - ❑ **Tiếp nhận** yêu cầu (HTTP Request) từ người dùng (do View gửi tới).
 - ❑ **Phân tích** yêu cầu.
 - ❑ **Yêu cầu** Model xử lý.
 - ❑ **Nhận** dữ liệu từ Model trả về.
 - ❑ **Lựa chọn** View thích hợp và **gửi** dữ liệu cho View đó để hiển thị.

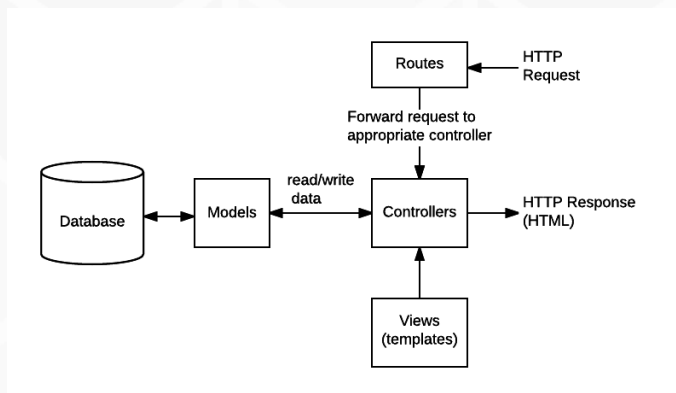
Ví dụ



Mô hình tương tác



Mô hình MVC với Express.js



Hỏi đáp



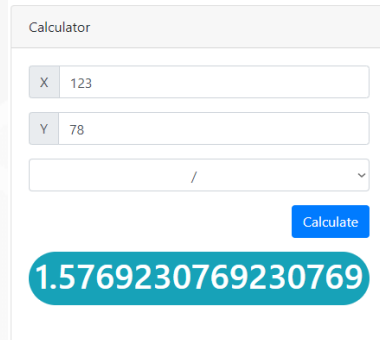
Bài tập

- ❑ Xây dựng ứng dụng Web “Bé tập tính” với express.js cho phép nhập vào 2 số x và y, lựa chọn phép tính và hiển thị kết quả (sử dụng response HTML).

A mockup of a web-based calculator interface. It has a title "Calculator" at the top. Below the title, there are two input fields: "X" with the value "40" and "Y" with the value "80". Below these is a dropdown menu showing a plus sign (+). To the right of the dropdown is a blue button labeled "Calculate". Below the button, the result "120" is displayed inside a green circle.

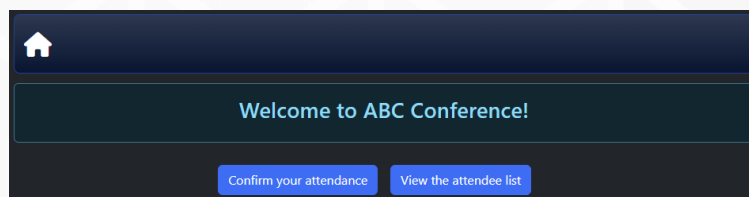
Bài tập

- ❑ Xây dựng ứng dụng Web “Bé tập tính” với express.js cho phép nhập vào 2 số x và y, lựa chọn phép tính và hiển thị kết quả (sử dụng AJAX).



Bài tập

- ❑ Xây dựng ứng dụng web giúp ghi nhận và phản hồi về việc đăng ký tham gia Hội nghị với các chức năng:
 - ❑ Trang chủ giới thiệu thông tin về Hội nghị
 - ❑ Trang xác nhận tham gia Hội nghị với các thông tin: Họ tên, Email, Điện thoại và xác nhận tham gia hay không.
 - ❑ Trang phản hồi kết quả.





Name

Email address

☐ Will you attend?

Submit



Thank you, abc, for providing feedback



List of feedback

#	Name	Email	Attend
1	Trần Văn A	tva@gmail.com	yes
2	Nguyễn Thị B	ntb@gmail.com	no
3	Phan Văn C	pvc@gmail.com	yes
4	Lê Thị D	ltd@gmail.com	yes
5	abc	abc@g.com	yes

Go home